A Smart Irrigation System Using Soil Moisture Sensing and Hysteresis Control

Somisetti Venkat Anish
M.Tech Scholar
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Andhra Pradesh, India
2601150089@kluniversity.in

Dr Raju Anitha
Professor
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Andhra Pradesh, India
anitharaju@kluniversity.in

***Abstract*—Fixed-schedule irrigation can apply too much or too little water because it ignores the actual state of the soil. This paper presents the design of a low-cost smart irrigation system in which an ESP32 microcontroller reads a capacitive soil moisture sensor, an air temperature and humidity sensor, and a tank water-level sensor, and switches a pump through a relay. Irrigation is controlled by an on-off law with hysteresis (separate lower and upper moisture thresholds), protected by safety interlocks for invalid sensor readings, an empty tank, and excessive pump run time. Telemetry is published over MQTT to a web or mobile dashboard that also accepts remote overrides and threshold updates. The paper describes the architecture, calibration, control algorithm, and communication design. Because field measurements are not yet available, the control logic is evaluated in a discrete-time soil-water simulation with stated, assumed parameters: over three simulated days, the threshold controller used 120 pump-on minutes against 260 for a fixed twice-daily schedule while keeping moisture inside the target band for 98.8% of the time. These are simulation results, not field measurements; a field-evaluation protocol is provided to replace them.**

***Index Terms*—Smart irrigation, Internet of Things, ESP32, soil moisture sensor, MQTT, hysteresis control, precision agriculture, water conservation.**

I. INTRODUCTION

Irrigation is one of the main ways in which farmers and gardeners protect crop yield against uneven rainfall. In many small installations, water is applied by hand or on a fixed timer, regardless of how wet the soil already is. A fixed schedule can therefore over-irrigate after rain, wasting water and energy, and under-irrigate during hot, dry spells. Smart water management based on the Internet of Things (IoT) addresses this by measuring the field and acting on the measurements [1]. Recent surveys review the sensors, actuators, processing nodes, and wireless technologies used in IoT irrigation systems [2] and the role of wireless sensor networks in irrigation [3].

A practical smart irrigation system needs three things: reliable sensing of soil water status, an actuation path that can switch a pump safely, and a communication path that lets the user observe and override the system. Low-cost microcontrollers with built-in Wi-Fi make all three affordable, which is why ESP32-based designs are common in the recent literature [4], [5].

This paper presents the design of an ESP32-based smart irrigation system. Its contributions are: (i) a modular hardware and software architecture built from inexpensive components; (ii) a hysteresis control law with explicit safety interlocks that protect the pump and the sensing chain; (iii) an MQTT-based telemetry and override interface [6]; and (iv) a simulation of the control logic and a field-evaluation protocol that separates what has been simulated from what remains to be measured. The system is presented as a reproducible baseline, not as a claim of superiority over other irrigation controllers.

The remainder of the paper is organized as follows. Section II reviews related work. Section III presents the theoretical background. Section IV describes the proposed system. Section V outlines the experimental setup. Section VI discusses the results. Section VII identifies limitations, and Section VIII concludes the paper and presents future work.

II. RELATED WORK

A. IoT-Based Irrigation Systems

At platform scale, the SWAMP project developed an IoT-based smart water management platform for precision irrigation and reports deployments in four pilots in Brazil and Europe, together with a performance analysis of the FIWARE components it uses [1]. The ARETHOU5A platform presents the subsystems and architecture of an intelligent irrigation system for precision agriculture, including an IoT node that can be powered by radio-frequency energy harvesting [8].

At device scale, Thote et al. describe a system built around an ESP32 board, a DHT11 humidity sensor, a soil moisture sensor, and relay-driven motors for automated irrigation and fertilization, with machine-learning support [4]. Abba et al. design a low-cost autonomous sensor interface for an IoT-based irrigation monitoring and control system, using an Arduino environment and a pumping mechanism to supply water from a reservoir [7]. Liao et al. develop smart irrigation systems driven by real-time soil moisture data in a greenhouse as a proof of concept [9]. The design in this paper belongs to this device-scale group: a single-board controller with a small set of sensors and a relay-driven pump.

B. Irrigation Scheduling Approaches

Irrigation scheduling can follow a calendar, a soil-moisture threshold, or a crop water balance. The FAO guidelines for computing crop water requirements estimate crop evapotranspiration from weather data and crop coefficients [10]; this is powerful but needs weather inputs and crop parameters. Soil-moisture-threshold control is simpler: it needs only a calibrated sensor and two set points, and it reacts directly to rainfall and drying because the soil itself is measured, as in the soil-moisture-driven systems of [7] and [9]. This paper adopts threshold control with hysteresis and leaves evapotranspiration-based scheduling to future work.

C. Enabling Technologies

The ESP32 is a Wi-Fi-enabled microcontroller with an on-chip analog-to-digital converter and general-purpose I/O pins [5]. MQTT is a lightweight publish/subscribe messaging protocol suited to constrained devices and unreliable networks [6]. Eclipse Mosquitto is an open-source broker that implements the protocol and is suitable for devices ranging from single-board computers to full servers [11]. Together they allow a field node to publish telemetry and subscribe to commands with little code and memory.

D. Positioning of the Proposed Work

The proposed system combines hysteresis moisture control, explicit fault handling, and a documented MQTT interface in one small design. Its distinguishing feature is the separation between simulated evidence (Section VI-A) and a protocol for field evidence (Section VI-B), so that the paper does not present modeled behavior as measured performance.

III. THEORETICAL BACKGROUND

A. Soil Water and Irrigation Thresholds

Plants take up water held in the soil between two limits: field capacity, above which excess water drains away, and the permanent wilting point, below which plants cannot extract water [10]. Irrigation should keep the volumetric moisture between a lower threshold above the wilting point and an upper threshold at or below field capacity. The thresholds depend on soil type and crop, so they are configurable parameters in this design rather than fixed constants.

B. Sensor Calibration

A capacitive soil moisture sensor produces an analog voltage that falls as moisture rises. The firmware averages N ADC samples to reduce noise and converts the mean reading to a relative moisture percentage using two calibration points: the reading in dry soil (or air) and the reading in saturated soil:

$$\bar{R} = (1/N)\ \Sigma_{j=1..N}\ R_j$$

$$m = 100 \cdot (R_{dry} - \bar{R}) / (R_{dry} - R_{wet}),\quad 0 \le m \le 100$$

The value m is a relative index, not an absolute volumetric water content; it must be recalibrated for each soil type.

C. Hysteresis (On-Off) Control

A single threshold would make the pump chatter on and off when the moisture hovers near the set point. Hysteresis avoids this by using two thresholds, a standard remedy for on-off controllers [12]. The pump command u(k) at step k is

$$u(k) = 1 \text{ if } m(k) < T_{low};\ \ 0 \text{ if } m(k) > T_{high};\ \ u(k-1) \text{ otherwise}$$

The band between T_{low} and T_{high} sets how much water is applied per irrigation event and how often events occur.

D. Simplified Soil-Water Balance

To test the controller without hardware, the root-zone moisture is modeled as a discrete-time water balance:

$$m(k+1) = m(k) + \rho \cdot u(k) + r(k) - e(k) - d(k)$$

where ρ is the moisture gain per step while the pump is on, r(k) is rainfall gain, e(k) is evapotranspiration loss, and d(k) is drainage loss. This is a deliberately simple model; it is used to exercise the control logic, not to predict a particular field.

IV. PROPOSED SYSTEM

A. Methodology

The system follows a sense–decide–act–report loop. At each sampling period the controller (1) reads the sensors, (2) filters and converts the readings, (3) checks the safety conditions, (4) applies the hysteresis law, (5) drives the relay, and (6) publishes telemetry and processes any incoming command; the full flow is given in Section IV-C. Because the loop only needs the local sensor values, irrigation continues to work if Wi-Fi is lost; only reporting and remote override are affected.

B. Hardware Architecture

The architecture has a sensing layer, an edge controller, an actuation path, and a network and application layer (Fig. 1). The components and their roles are listed in Table I, and a typical pin mapping is shown in Fig. 2. The pump is powered from a separate supply and switched through a relay so that the pump's current never flows through the microcontroller board.

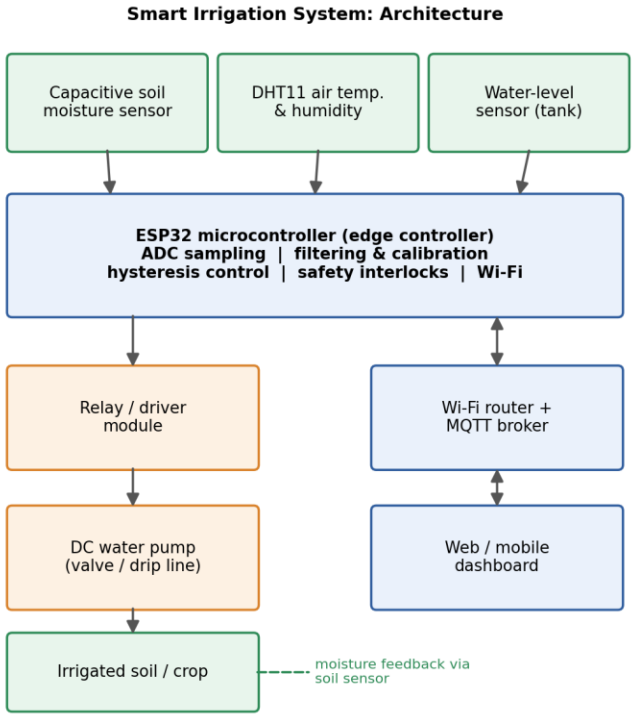


Fig. 1. Architecture of the smart irrigation system.

Table 1 : Hardware Componets Roles

Component	Role in the system
ESP32 DevKit	Edge controller: ADC sampling, control logic, Wi-Fi and MQTT client.
Capacitive soil moisture sensor	Analog output related to soil moisture; main control input.
DHT11 sensor	Air temperature and relative humidity for monitoring and context.
Water-level sensor	Detects an empty tank to prevent dry running of the pump.
5 V relay module	Switches the pump supply under ESP32 control.
DC water pump	Delivers water through a valve or drip line to the soil.

Component	Role in the system
Wi-Fi router, MQTT broker	Network path and message broker for telemetry and commands.
Web / mobile dashboard	Displays readings and sends overrides and threshold updates.

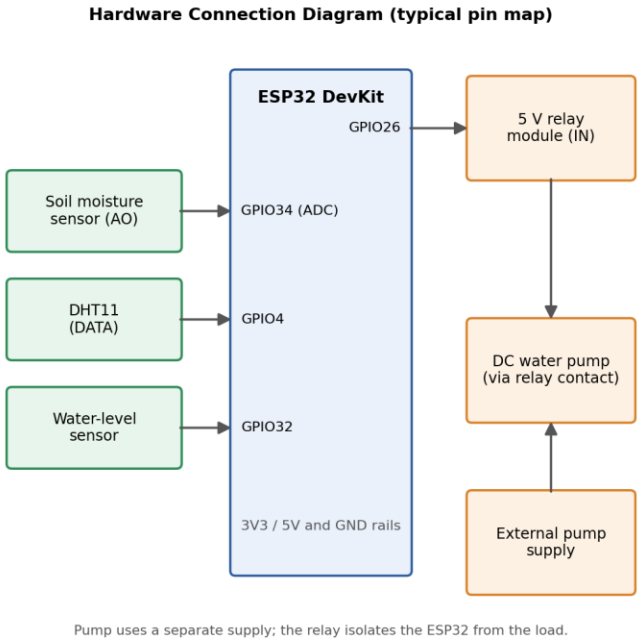


Fig. 2. Hardware connection diagram with a typical pin map (pins are configurable).

C. Control Algorithm and Safety Interlocks

The control flow is shown in Fig. 3 and summarized in Algorithm 1. Three interlocks run before the hysteresis law so that a fault always forces the pump off: (1) an invalid sensor reading (for example, a value outside the calibrated range, which indicates a disconnected or failed sensor) forces the pump off and raises an alert; (2) a low tank level forces the pump off to prevent dry running; and (3) a maximum continuous run time t_{max} stops the pump even if the upper threshold has not been reached, which limits the damage from a stuck sensor or a leak.

Algorithm 1: Threshold Irrigation Control with Safety Interlocks

Input: Thresholds T_low, T_high; sampling period T_s; maximum run time t_max
Output: Pump command u and telemetry
1: Initialize Wi-Fi, MQTT client, constants R_dry, R_wet; set u ← 0
2: Repeat every T_s:
3: Read N ADC samples; compute m using (1) and (2)
4: If m is invalid or tank level is low: u ← 0, publish alert, go to 9
5: If m < T_low: u ← 1 and start the run timer
6: Else if m > T_high: u ← 0
7: If u = 1 and run time > t_max: u ← 0, publish alert
8: Drive the relay with u
9: Publish m, air temperature, humidity, u and tank status
10: Apply any remote override or new thresholds received

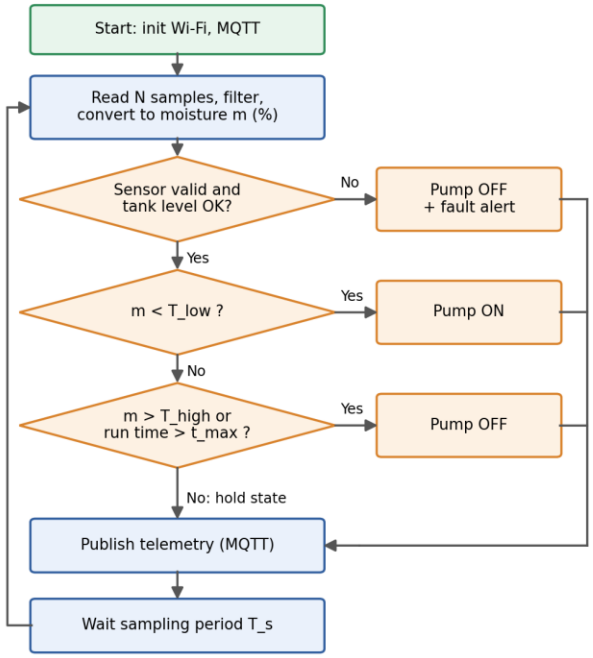


Fig. 3. Control flow of the irrigation controller.

D. Communication and Dashboard

The node publishes telemetry (moisture, air temperature, humidity, pump state, tank status, and alerts) as a JSON message to a per-zone MQTT topic [6]. It subscribes to a command topic for manual pump control and to a configuration topic for new thresholds. A manual override takes effect through the same safety interlocks, so a remote command cannot run the pump against an empty tank. The dashboard subscribes to the telemetry topic and shows current values, history, and the pump state. For deployment beyond a laboratory network, the broker connection should use authentication and TLS.

E. Tools and Technologies

The firmware targets the ESP32 and uses the Arduino framework with an MQTT client library for Wi-Fi communication. The dashboard can be any MQTT-capable front end, and a Mosquitto broker [11] is sufficient for a single-zone prototype. The design uses only widely available components and open protocols.

V. EXPERIMENTAL SETUP

A. Implementation Environment

The experimental focus is functional correctness and safety: the pump must turn on below T_{low}, off above T_{high}, and off whenever an interlock fires. Hardware testing uses the components in Table I with the sensor calibrated in dry and saturated soil of the target type.

B. Simulation Setup

Because field data are not yet available, the control logic was tested in the discrete-time model of (4). One step represents 10 minutes, and the horizon is three days. The threshold controller (hysteresis, Table II) is compared with a fixed schedule of 40 minutes of pumping at 06:00 and 18:00 each day. Evapotranspiration follows a half-sine over 06:00–18:00, and one rainfall event occurs on the second day. All parameters in Table II are assumptions chosen to exercise the logic; they are not measured values. The simulated run does not exercise the tank-level and run-time interlocks.

Table 2 : Simulation Parameter

Parameter	Value
Control step / horizon	10 min / 3 days
Initial moisture	50 %
Thresholds T_low, T_high	35 %, 55 %
Pump gain ρ (pump on)	1.5 points per step
Peak evapotranspiration e	0.30 points per step
Rain event	Day 2, 10:00–11:00, +15 points
Drainage above 60 %	50 % of excess per step
Fixed-schedule baseline	40 min at 06:00 and 18:00

C. Field Evaluation Protocol

The metrics in Table IV define the measurements to be taken once the prototype is running. They are listed so that the simulated figures in Section VI-A can be replaced by measured ones.

VI. RESULTS AND DISCUSSION

A. Simulation Results

Fig. 4 shows the simulated moisture under both strategies, and Table III summarizes the three-day run. In this scenario the threshold controller used 120 pump-on minutes against 260 for the fixed schedule, about 54% fewer, because it did not irrigate when moisture was already adequate and benefited from the rain event. It also lost less water to drainage (3.5 against 10.0 moisture points) and kept moisture within 35–60% for 98.8% of the time against 97.2%. Neither strategy fell below 30%. The threshold run touched the lower threshold (minimum 34.9%) because the pump had to start before the rain arrived. The difference is mainly attributable to demand-based operation: the controller responds to the measured moisture state instead of applying water at predetermined times. After the rainfall event, the moisture level rises and the threshold policy can postpone further irrigation until the lower limit is approached again. The fixed schedule, by contrast, continues to apply water at its programmed times even when recent rainfall has already increased soil moisture. The peak value of 61.6% in both runs also indicates that the simplified model and the selected event timing can produce a temporary rise above the 60% target-band boundary. Therefore, the results should be interpreted as a comparison of control behavior under one defined scenario rather than as a universal estimate of water savings.

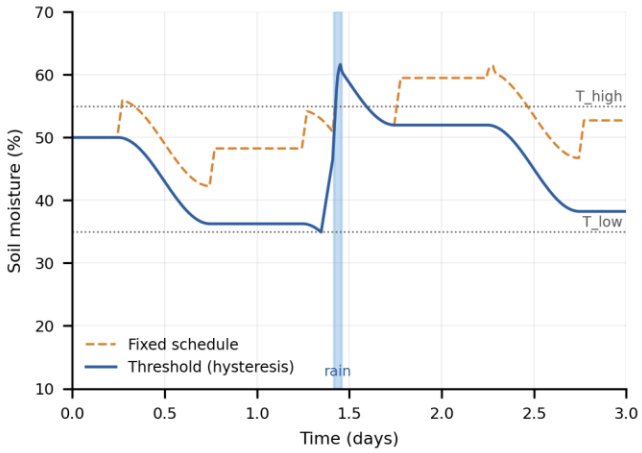


Fig. 4. Simulated soil moisture under threshold (hysteresis) control and a fixed schedule.

Table 3 :Simualtion Result

Metric	Fixed schedule	Threshold
Pump-on time (min)	260	120
Drainage loss (points)	10.0	3.5
Minimum moisture (%)	42.3	34.9
Maximum moisture (%)	61.6	61.6
Time below 30 % (min)	0	0
Time within 35–60 % (%)	97.2	98.8

These figures come from the assumed model in Table II. They show that the control logic behaves as designed, but they do not establish the water savings that a real field would achieve.

B. Planned Field Evaluation

Table IV lists the measurements planned for the prototype. The right-hand column is intentionally left to be completed with measured data. During field testing, each observation should be time-stamped and collected under comparable soil, crop, and weather conditions. The fixed-schedule baseline and threshold controller should be tested over equivalent periods, with rainfall and any manual intervention recorded. Repeated trials would help distinguish consistent controller behavior from variations caused by sensor placement, soil heterogeneity, or changing weather. Only after these measurements are available should the simulated reductions in pump operation and drainage be compared with actual water consumption and plant response.

Table 4: Table Evalatuion

Metric	Method	Result
Sensor calibration error	Compare m with oven-dry gravimetric samples	To be recorded
Actuation latency	Time from reading to relay change	To be recorded
Telemetry delivery	MQTT messages received vs. sent	To be recorded
Water used per day	Pump-on time × flow rate vs. baseline timer	To be recorded
Time in target band	Logged m between T_low and T_high	To be recorded
Fault handling	Sensor unplugged, empty tank, Wi-Fi loss	To be recorded

C. Discussion

The design keeps the decision on the device, so irrigation does not depend on the network. Hysteresis prevents pump chatter, and the interlocks make failures end in a safe state (pump off). Threshold control is easy to configure and explain, but it is reactive: it does not anticipate weather, and the right thresholds depend on soil and crop. The simulation supports the control logic, while the real benefit of the system must be established by the field evaluation in Table IV.

VII. LIMITATIONS

First, no field measurements are reported; all quantitative results come from a simplified simulation whose parameters are assumptions. Second, the moisture value is a relative index that depends on per-soil calibration and sensor placement, and a single sensor may not represent a whole plot. Third, the low-cost air sensor limits the accuracy of the temperature and humidity readings. Fourth, the system controls one zone and uses threshold control without weather forecasting. Fifth, a basic MQTT setup without authentication and TLS is not secure for field deployment. Finally, the pump delivery rate is not measured, so water volume is estimated from run time.

VIII. CONCLUSION AND FUTURE WORK

This paper presented an ESP32-based smart irrigation system that combines capacitive soil moisture sensing, hysteresis control, safety interlocks, and MQTT telemetry and override. In a simulated three-day scenario with assumed parameters, the controller used fewer pump-on minutes than a fixed schedule while keeping moisture in the target band. The design emphasizes low cost, safe failure behavior, and a clear separation between simulated and measured evidence.

Future work will complete the field evaluation in Table IV; add a flow meter to measure delivered water; extend the controller to several zones with solenoid valves; add evapotranspiration-based scheduling using weather data [10]; power the node from a solar panel; and secure the MQTT link with authentication and TLS.

REFERENCES

[1] C. Kamienski et al., “Smart water management platform: IoT-based precision irrigation for agriculture,” Sensors, vol. 19, no. 2, Art. no. 276, 2019, doi: 10.3390/s19020276.

[2] L. García, L. Parra, J. M. Jimenez, J. Lloret, and P. Lorenz, “IoT-based smart irrigation systems: An overview on the recent trends on sensors and IoT systems for irrigation in precision agriculture,” Sensors, vol. 20, no. 4, Art. no. 1042, 2020, doi: 10.3390/s20041042.

[3] L. Hamami and B. Nassereddine, “Application of wireless sensor networks in the field of irrigation: A review,” Comput. Electron. Agric., vol. 179, Art. no. 105782, 2020, doi: 10.1016/j.compag.2020.105782.

[4] D. Thote, V. Lanjewar, V. Sharma, P. Agrawal, and V. K. Soni, “IoT and machine learning-based smart soil irrigation farming systems,” in Proc. IEEE Int. Students’ Conf. Elect., Electron. Comput. Sci. (SCEECS), 2024, doi: 10.1109/SCEECS61402.2024.10482246.

[5] Espressif Systems, “ESP32 Series Datasheet,” Espressif Systems, Shanghai, China. [Online]. Available: https://www.espressif.com

[6] A. Banks and R. Gupta, Eds., MQTT Version 3.1.1, OASIS Standard, Oct. 2014.

[7] S. I. Abba, J. W. Namkusong, J.-A. Lee, and M. L. Crespo, “Design and performance evaluation of a low-cost autonomous sensor interface for a smart IoT-based irrigation monitoring and control system,” Sensors, vol. 19, no. 17, Art. no. 3643, 2019, doi: 10.3390/s19173643.

[8] A. D. Boursianis et al., “Smart irrigation system for precision agriculture—The AREThOU5A IoT platform,” IEEE Sensors J., vol. 21, pp. 17539–17547, 2021.

[9] R. Liao, S. Zhang, X. Zhang, M. Wang, H. Wu, and L. Zhangzhong, “Development of smart irrigation systems based on real-time soil moisture data in a greenhouse: Proof of concept,” Agric. Water Manage., vol. 245, Art. no. 106632, 2021, doi: 10.1016/j.agwat.2020.106632.

[10] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements, FAO Irrigation and Drainage Paper 56. Rome, Italy: Food and Agriculture Organization of the United Nations, 1998.

[11] R. A. Light, “Mosquitto: Server and client implementation of the MQTT protocol,” J. Open Source Softw., vol. 2, no. 13, p. 265, 2017, doi: 10.21105/joss.00265.

[12] K. J. Åström and R. M. Murray, Feedback Systems: An Introduction for Scientists and Engineers. Princeton, NJ, USA: Princeton Univ. Press, 2008.